

NTL-SysToolbox

Documentation Technique

*Suite d'outils d'administration pour
l'infrastructure NordTransit Logistics*

MSPR TPRE511 - B3 ASRBD EPSI

Equipe : Quentin, Mathis, Eliot, Julien

Date de passage : 24 fevrier 2026

Source du projet :

<https://github.com/htipilou/MSPR-NTL>

Document fait le 14/02/2026

Version 2.0

Sommaire

1. Introduction et Contexte	3
1.1 Presentation du projet	3
1.2 Contexte NordTransit Logistics	3
1.3 Objectifs de la solution	3
2. Justification des Choix Techniques	4
2.1 Langage Python	4
2.2 Architecture modulaire	4
2.3 Technologies et librairies	5
2.4 Protocole de communication	5
3. Architecture de la Solution	6
3.1 Schema d'architecture	6
3.2 Composants principaux	6
3.3 Flux de donnees	7
4. Modules Fonctionnels	8
4.1 Diagnostic Infrastructure	8
4.2 Sauvegarde MySQL	9
4.3 Audit d'Obsolescence	10
4.4 Agent Systeme	11
5. Guide d'Installation	12
5.1 Prerequis	12
5.2 Installation pas a pas	12
5.3 Configuration	13
6. Guide d'Utilisation	14
6.1 Interface de selection	14
6.2 Utilisation des modules	14
6.3 Interpretation des resultats	15
7. Securite	16
7.1 Authentification	16
7.2 Bonnes pratiques	16
8. Maintenance et Support	17
9. Annexes	18

1. Introduction et Contexte

1.1 Presentation du projet

NTL-SysToolbox est une suite d'outils d'administration developpee pour repondre aux besoins de maintenance, de diagnostic et d'audit de l'infrastructure informatique de NordTransit Logistics. Cette solution permet a la DSI de superviser efficacement l'ensemble du parc serveurs et d'automatiser les taches critiques de maintenance.

1.2 Contexte NordTransit Logistics

NordTransit Logistics est une entreprise de logistique dont l'infrastructure IT comprend plusieurs serveurs critiques : controleurs de domaine Active Directory, serveurs de base de donnees MySQL, serveurs web applicatifs, et outils de supervision. La disponibilite de ces services est essentielle pour assurer la continuite des operations.

1.3 Objectifs de la solution

- Automatiser le diagnostic de l'infrastructure reseau et systeme
- Assurer la sauvegarde automatisee et securisee des bases de donnees
- Auditer le parc informatique pour identifier les systemes obsoletes (EOL)
- Collecter les metriques systeme en temps reel via des agents deployes
- Fournir des rapports exploitables au format JSON et CSV

2. Justification des Choix Techniques

2.1 Langage Python

Python a ete choisi comme langage principal pour plusieurs raisons strategiques :

- Portabilite : Fonctionne nativement sur Windows Server et Linux (Ubuntu, Debian, CentOS)
- Richesse de l'ecosysteme : Librairies matures pour le systeme (psutil), reseau (socket), et BDD (pymysql)
- Rapidite de developpement : Syntaxe claire permettant une maintenance aisee par la DSI
- Standard en administration systeme : Largement adopte pour les scripts d'automatisation
- Documentation abondante : Facilite la prise en main par de nouveaux membres de l'equipe

2.2 Architecture modulaire

L'architecture a ete concue de maniere modulaire pour permettre :

- Independance des modules : Chaque script peut etre execute separement
- Evolutivite : Ajout facile de nouveaux modules sans impact sur l'existant
- Testabilite : Tests unitaires isoles pour chaque composant
- Reutilisabilite : Composants partages (config_loader, agent_client)

2.3 Technologies et librairies

Composant	Technologie	Justification
Configuration	YAML + PyYAML	Format lisible, supporte les structures complexes
Metriques systeme	psutil	Cross-platform, API complete pour CPU/RAM/Disk
Base de donnees	PyMySQL	Client MySQL natif Python, pas de dependance C
Interface CLI	questionary	Interface interactive elegante et intuitive
Tests	pytest	Framework de test standard, riche en fonctionnalites

2.4 Protocole de communication Agent

Le protocole TCP a ete choisi pour la communication entre les agents et le serveur central :

- Fiabilité : TCP garantit la livraison des paquets contrairement à UDP
- Simplicité : Pas besoin d'un framework complexe (HTTP, gRPC)
- Performance : Overhead minimal pour des messages JSON simples
- Sécurité : Authentification par token configurable

3. Architecture de la Solution

3.1 Schema d'architecture

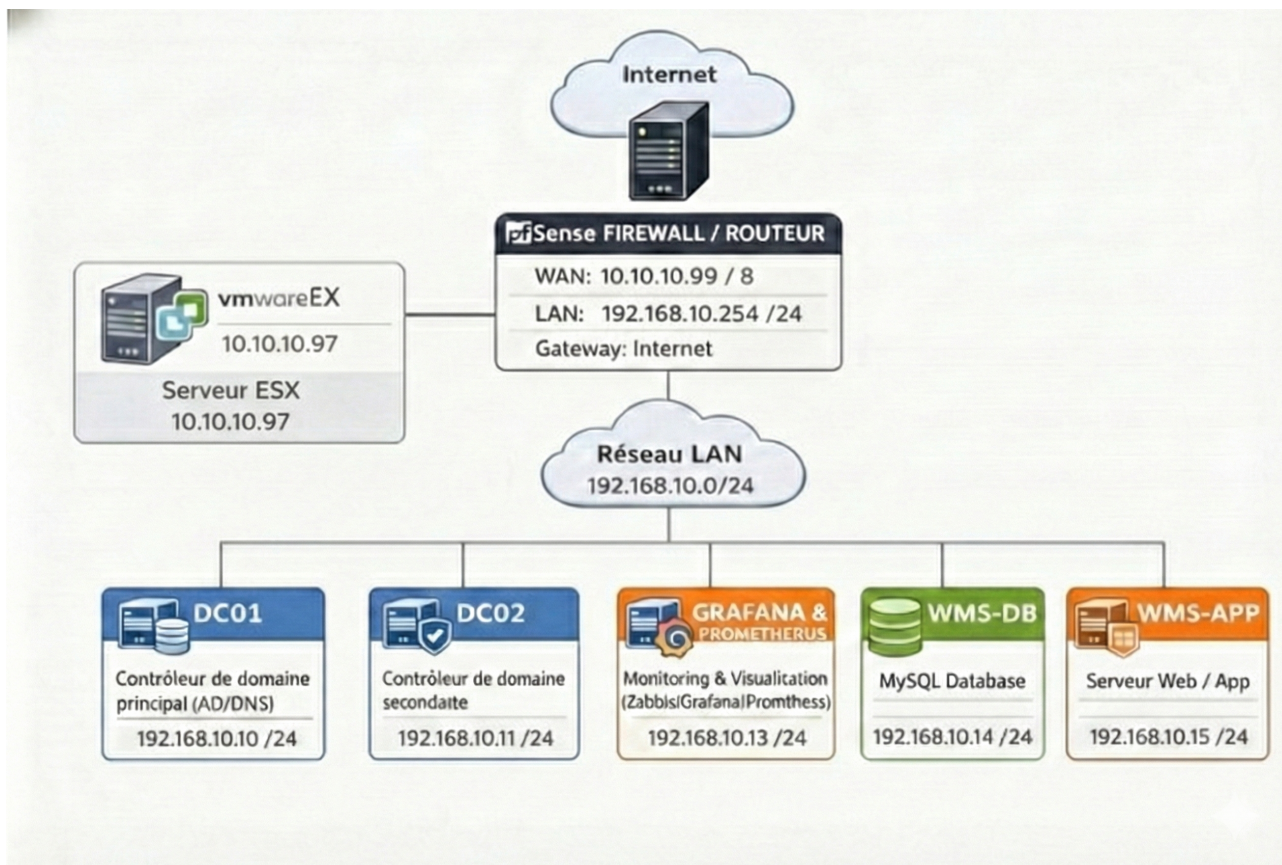


Tableau adressage IP :

Serveur	Rôle / Description	IP (LAN / WAN)	Masque	Passerelle (Gateway)
vmwareEX	Serveur ESX	10.10.10.97	/8	-
pfSense WAN	Interface externe	10.10.10.99	/8	Routeur amont
pfSense LAN	Passerelle réseau	192.168.10.254	/24	-
DC01	Contrôleur de domaine	192.168.1.10	/24	192.168.10.254
DC02	Contrôleur de domaine	192.168.1.11	/24	192.168.10.254
SUPER-01	Zabbix / Grafana	192.168.1.13	/24	192.168.10.254
WMS-DB	MySQL Database	192.168.1.14	/24	192.168.10.254
WMS-APP	Serveur Web / App	192.168.1.15	/24	192.168.10.254

3.2 Composants principaux

Fichier	Role	Description
selecteur.py	Interface	Menu interactif de selection des modules
diagnostique_infra.py	Diagnostic	Test connectivite et collecte metriques
backup_mysql.py	Sauvegarde	Export complet des bases MySQL
audit.py	Audit EOL	Analyse obsolescence du parc
ntl_agent.py	Agent	Daemon collecte metriques systeme
config_loader.py	Configuration	Chargement centralise YAML
agent_client.py	Client	Interrogation des agents distants

3.3 Infrastructure supportee

Serveur	IP	OS	Role
AD-01	192.168.10.10	Windows Server 2019	Controleur domaine principal
AD-02	192.168.10.11	Windows Server 2019	Controleur domaine secondaire
WMS-DB	192.168.10.14	Ubuntu 20.04 LTS	Base de donnees MySQL
WMS-APP	192.168.10.15	Ubuntu 20.04 LTS	Serveur Web applicatif
GRAFANA	192.168.10.13	Windows Server 2025	Supervision
PFSense	192.168.10.1	pfSense 2.7	Firewall LAN
ESXI	10.10.10.97	VMware ESXi	Hyperviseur

4. Modules Fonctionnels

4.1 Diagnostic Infrastructure (diagnostique_infra.py)

Ce module effectue un bilan de sante complet de l'infrastructure en testant la connectivite des ports critiques et en collectant les metriques systeme via les agents deployes.

Fonctionnalites

- Test de connectivite TCP sur les ports critiques (SSH, HTTP, HTTPS, LDAP, MySQL, RDP)
- Verification de la connexion applicative aux bases de donnees MySQL
- Collecte des metriques CPU, RAM, Disque via les agents systeme
- Generation de rapports JSON horodates dans /rapports_ntl/
- Codes de retour exploitables pour l'automatisation (0=OK, 1=Warning, 2=Critical)
- Ici on a changé le WMS-DB de 192.168.10.14 à 192.168.10.250 et on peut le voir qu'il est bien détecté
- On a aussi testé d'éteindre l'agent sur la 2ème AD et on voit que le rapport nous indique que l'agent n'est pas joignable

```

? Que souhaitez-vous faire ? Diagnostic Infrastructure (diagnostic_infra.py)
--- Lancement de diagnostic_infra.py ---

=====
DIAGNOSTIC AUTO INFRASTRUCTURE NTL - COMPLET
=====

[*] Test ESXi Host (10.10.10.97)...
[*] Test ESXi Host (10.10.10.97)...
  [*] Verification agent sur port 6000...
  [!] Agent DEBUG: commande debug indisponible (ancienne version?)
  [*] Interrogation agent sur port 6000...
  [!] Agent: Failed to retrieve metrics
[*] Test Grafana (192.168.10.13)...
[*] Test Grafana (192.168.10.13)...
  [*] Verification agent sur port 6000...
  [✓] Agent DEBUG: v1.1.0 | port=6000 | socket=True | metriques=True
  [*] Interrogation agent sur port 6000...
  [✓] Agent: Métriques système collectées et validées
[*] Test WMS-DB (192.168.10.250)...
[*] Test WMS-DB (192.168.10.250)...
  [*] Verification agent sur port 6000...
  [✓] Agent DEBUG: v1.1.0 | port=6000 | socket=True | metriques=True
  [*] Interrogation agent sur port 6000...
  [✓] Agent: Métriques système collectées et validées
[*] Test Serveur AD 1 (192.168.10.10)...
[*] Test Serveur AD 1 (192.168.10.10)...
  [*] Verification agent sur port 6000...
  [✓] Agent DEBUG: v1.1.0 | port=6000 | socket=True | metriques=True
  [*] Interrogation agent sur port 6000...
  [✓] Agent: Métriques système collectées et validées
[*] Test Serveur AD MSPR (192.168.10.11)...
[*] Test Serveur AD MSPR (192.168.10.11)...
  [*] Verification agent sur port 6000...
  [!] Agent DEBUG: commande debug indisponible (ancienne version?)
  [*] Interrogation agent sur port 6000...
  [!] Agent: Failed to retrieve metrics
[*] Test WMS-APP (192.168.10.15)...
[*] Test WMS-APP (192.168.10.15)...
  [*] Verification agent sur port 6000...
  [✓] Agent DEBUG: v1.1.0 | port=6000 | socket=True | metriques=True
  [*] Interrogation agent sur port 6000...
  [✓] Agent: Métriques système collectées et validées

=====
RÉSUMÉ DIAGNOSTIC
=====
Serveurs testés: 6
✓ OK: 3 | ▲ Alertes: 3 | ✗ Down: 0

=====
▲ ESXi Host      (10.10.10.97) - 1/2 ports OK | Fermés: SSH 22 | Agent: Indisponible
▲ Grafana        (192.168.10.13) - 1/2 ports OK | Fermés: SSH 22 | Agent: OK (CPU:2.3% RAM:67.1%)
✓ WMS-DB         (192.168.10.250) - 3/3 ports OK | Agent: OK (CPU:4.0% RAM:33.9%)
✓ Serveur AD 1   (192.168.10.10) - 5/5 ports OK | Agent: OK (CPU:0.0% RAM:76.4%)
✓ Serveur AD MSPR (192.168.10.11) - 5/5 ports OK | Agent: Indisponible
▲ WMS-APP        (192.168.10.15) - 2/3 ports OK | Fermés: HTTPS 443 | Agent: OK (CPU:0.0% RAM:17.8%)

=====
[✓] Rapport complet sauvegardé: rapports_ntl/diagnostic_20260213_201037.json
=====

[Info] Diagnostic terminé avec alertes (code 1).
Appuyez sur Entrée pour revenir au menu...

```

```

},
"Grafana": {
  "ip": "192.168.10.13",
  "name": "Grafana",
  "grafana_3000": true,
  "ssh_22": false,
  "agent_debug": {
    "agent_version": "1.1.0",
    "bind_address": "0.0.0.0",
    "port": 6000,
    "actual_bind": "0.0.0.0:6000",
    "socket_listening": true,
    "auth_enabled": false,
    "agent_uptime_seconds": 176566.1,
    "agent_start_time": "2026-02-11T18:02:20.541701",
    "connections_handled": 19,
    "metrics_collection_ok": true,
    "metrics_collection_error": null,
    "metrics_sample": {
      "hostname": "WIN-JBHJGFPAJU4",
      "cpu_percent": 1.6,
      "ram_percent": 67.1,
      "partitions_count": 1
    }
  },
  "timestamp": "2026-02-13T19:05:06.603298"
},
"agent": {
  "timestamp": "2026-02-13T19:05:07.608928",
  "hostname": "WIN-JBHJGFPAJU4",
  "os": {
    "system": "Windows",
    "release": "10",
    "version": "10.0.26100",
    "architecture": "AMD64",
    "platform": "Windows-10-10.0.26100-SP0"
  },
  "cpu": {
    "usage_percent": 2.3,
    "count_logical": 2,
    "count_physical": 2,
    "frequency_mhz": {
      "current": 2295.0,
      "min": 0.0,
      "max": 2295.0
    }
  },
  "memory": {
    "total_mb": 4095.03,
    "available_mb": 1346.46,
    "used_mb": 2748.57,
    "percent": 67.1,
    "swap_total_mb": 704.0,
    "swap_used_mb": 258.58,
    "swap_percent": 36.7
  },
  "disk": {
    "partitions": [
      {
        "device": "C:\\",
        "mountpoint": "C:\\",
        "fstype": "NTFS",
        "total_gb": 98.33,
        "used_gb": 16.83,
        "free_gb": 81.51,
        "percent": 17.1
      }
    ]
  },
  "uptime": {
    "boot_time": "2026-02-11T17:23:54.885974",
    "uptime_seconds": 178873,
    "uptime_human": "2 days, 1:41:13"
  },
  "agent_validation": {
    "ok": true,
    "errors": []
  }
}

```

Aide Écrire Chercher Couper Exécuter Emplacement M-II App

4.2 Sauvegarde MySQL (backup_mysql.py)

Module de sauvegarde automatisee des bases de donnees MySQL avec export complet incluant les routines, triggers et evenements.

Fonctionnalites

- Export complet avec --all-databases
- Mode non-bloquant avec --single-transaction
- Inclusion des routines, triggers et evenements
- Rapport JSON de sauvegarde avec taille et horodatage
- Configuration securisee via config.yaml (pas de mot de passe en dur)

```
? Que souhaitez-vous faire ? Sauvegarde MySQL (backup_mysql.py)
--- Lancement de backup_mysql.py ---

=====
BACKUP MYSQL COMPLET AUTO
Serveur: 192.168.10.250
=====

[*] Lancement backup complet...
[*] Fichier: backups_mysql/backup_complet_20260213_190459.sql

=====
✓ BACKUP RÉUSSI
=====
Fichier: backups_mysql/backup_complet_20260213_190459.sql
Taille: 2.42 MB
Rapport: backups_mysql/rapport_20260213_190459.json
=====

Appuyez sur Entrée pour revenir au menu...█
```

4.3 Audit d'Obsolescence (audit.py)

Outil de gestion du cycle de vie du parc informatique permettant d'identifier les systemes en fin de vie (EOL) et d'evaluer les risques associes.

Niveaux de risque

Niveau	Indicateur	Description
CRITIQUE	Rouge	Système obsolète, plus de support sécurité
ELEVE	Orange	Fin de support dans moins de 6 mois
MOYEN	Jaune	En support étendu uniquement
FAIBLE	Bleu clair	Support étendu disponible
OK	Vert	Support principal actif

[✓] Scan terminé: 6 hôte(s) actif(s)

=====

RAPPORT D'AUDIT D'OBSOLESCENCE - NORDTRANSIT LOGISTICS

=====

Date: 2026-02-13 20:08:03

Infrastructure scannée: 6 hôte(s)

=====

📋 SYNTHÈSE PAR NIVEAU DE RISQUE

- CRITIQUE : 0 hôte(s) - EOL dépassée - ACTION IMMÉDIATE
 - ÉLEVÉ : 0 hôte(s) - < 6 mois - PLANIFICATION URGENTE
 - MOYEN : 0 hôte(s) - < 1 an - PLANIFICATION
 - FAIBLE : 0 hôte(s) - < 2 ans - SURVEILLER
 - ☑ OK : 4 hôte(s) - Support actif
 - INCONNU : 2 hôte(s) - Version non identifiée
- =====

☑ NIVEAU OK (4 hôte(s))

🔍 N/A (192.168.10.10)

OS: Windows Server 2019
Rôle: Non documenté
Version EOL: Windows Server 2019
Extended EOL: 2029-01-09
Jours restants: 1060
Recommandation: Support actif - OK

🔍 N/A (192.168.10.15)

OS: Ubuntu 20.04 LTS
Rôle: Non documenté
Version EOL: Ubuntu 20.04 LTS
Extended EOL: 2030-04-02
Jours restants: 1508
Recommandation: Support actif - OK

🔍 N/A (192.168.10.11)

OS: Windows Server 2019
Rôle: Non documenté
Version EOL: Windows Server 2019
Extended EOL: 2029-01-09
Jours restants: 1060
Recommandation: Support actif - OK

🔍 N/A (192.168.10.250)

OS: Ubuntu 20.04 LTS
Rôle: Non documenté
Version EOL: Ubuntu 20.04 LTS
Extended EOL: 2030-04-02
Jours restants: 1508
Recommandation: Support actif - OK

○ NIVEAU UNKNOWN (2 hôte(s))

🔍 N/A (192.168.10.13)

OS: Windows Server
Rôle: Non documenté
Version EOL: Version inconnue
Extended EOL: N/A
Recommandation: Vérifier manuellement

🔍 N/A (192.168.10.254)

OS: Système inconnu
Rôle: Non documenté
Version EOL: Version inconnue
Extended EOL: N/A
Recommandation: Vérifier manuellement

=====

[✓] Export CSV sauvegardé: audit_eol_ntl_20260213_200803.csv

```
GNU nano 8.4 audit_eol_ntl_20260213_200803.csv
#0,hostname,os,role,source,matched_version,eol_status,mainstream_eol,extended_eol,days_remaining,risk_level,recommendation
192.168.10.10,N/A,Windows Server 2019,Non documenté,detected,Windows Server 2019,Extended Support,2024-01-09,2029-01-09,1060,OK,Support actif - OK
192.168.10.15,N/A,Ubuntu 20.04 LTS,Non documenté,detected,Ubuntu 20.04 LTS,Mainstream Support,2025-04-02,2030-04-02,1508,OK,Support actif - OK
192.168.10.13,N/A,Windows Server,Non documenté,detected,Version inconnue,UNKNOWN,N/A,N/A,N/A,UNKNOWN,Vérifier manuellement
192.168.10.11,N/A,Windows Server 2019,Non documenté,detected,Windows Server 2019,Extended Support,2024-01-09,2029-01-09,1060,OK,Support actif - OK
192.168.10.250,N/A,Ubuntu 20.04 LTS,Non documenté,detected,Ubuntu 20.04 LTS,Mainstream Support,2025-04-02,2030-04-02,1508,OK,Support actif - OK
192.168.10.254,N/A,Système inconnu,Non documenté,detected,Version inconnue,UNKNOWN,N/A,N/A,N/A,UNKNOWN,Vérifier manuellement
```

4.4 Agent Systeme (ntl_agent.py)

Daemon léger deployé sur chaque serveur pour exposer les métriques système via un protocole TCP sécurisé. L'agent collecte en temps réel les informations CPU, RAM, Disque et Uptime.

Métriques collectées

Metrique	Donnees	Unite
CPU	Usage, frequence, nombre de coeurs	%, MHz
RAM	Total, utilise, disponible, pourcentage	MB, %
Disque	Total, utilise, libre par partition	GB, %
Uptime	Duree depuis dernier demarrage	jours, heures
OS	Version, architecture, hostname	texte

```
(venv) root@web:/home/web/MSPR-NTL# systemctl status ntl_agent.service
● ntl_agent.service - Agent de Monitoring NTL
   Loaded: loaded (/etc/systemd/system/ntl_agent.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-02-13 19:49:11 CET; 17min ago
  Invocation: 4f0fe81086f341e69b520773713b0d90
    Main PID: 14187 (python)
      Tasks: 1 (limit: 2301)
     Memory: 8.6M (peak: 10.3M)
        CPU: 111ms
    CGroup: /system.slice/ntl_agent.service
            └─14187 /home/web/MSPR-NTL/venv/bin/python /home/web/MSPR-NTL/ntl_agent.py

févr. 13 19:49:11 web systemd[1]: ntl_agent.service: Scheduled restart job, restart counter is at 31.
févr. 13 19:49:11 web systemd[1]: Started ntl_agent.service - Agent de Monitoring NTL.
(venv) root@web:/home/web/MSPR-NTL#
```

5. Guide d'Installation

5.1 Prerequis

Composant	Version minimum	Notes
Python	3.13+	Inclus sur Ubuntu 20.04+
pip	20.0+	Gestionnaire de paquets Python
mysql-client	5.7+	Pour les sauvegardes BDD
Git	2.0+	Pour cloner le depot

5.2 Installation pas a pas

Etape 1 : Cloner le depot

```
git clone https://github.com/chtipilou/MSPR-NTL.git
cd MSPR-NTL
```

Etape 2 : Installer les dependances

```
pip3 install -r requirements.txt
```

```
# Remove the broken venv directory
rm -rf venv

# Create a new one
python3 -m venv venv

# Reactivate it
source venv/bin/activate

# Reinstall your dependencies
pip install -r requirements.txt

Collecting pyyaml==6.0 (from -r requirements.txt (line 2))
  Using cached pyyaml-6.0.3-cp310-cp313-manylinux2014_x86_64-manylinux_2_17_x86_64-manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting psutil==5.9.0 (from -r requirements.txt (line 3))
  Using cached psutil-7.2.2-cp38-ab33-manylinux2010_x86_64-manylinux_2_12_x86_64-manylinux_2_28_x86_64.whl.metadata (22 kB)
Collecting pymysql==1.0.0 (from -r requirements.txt (line 4))
  Using cached pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
Collecting questionnaire==2.0.0 (from -r requirements.txt (line 5))
  Using cached questionnaire-2.1.1-py3-none-any.whl.metadata (5.4 kB)
Collecting pytest==7.0.0 (from -r requirements.txt (line 6))
  Using cached pytest-9.0.2-py3-none-any.whl.metadata (7.0 kB)
Collecting pytest-cov==4.0.0 (from -r requirements.txt (line 7))
  Using cached pytest-cov-7.0.0-py3-none-any.whl.metadata (31 kB)
Collecting prompt_toolkit==4.0.0 (from questionnaire==2.0.0->-r requirements.txt (line 5))
  Using cached prompt_toolkit-3.0.52-py3-none-any.whl.metadata (6.4 kB)
Collecting wcwidth (from prompt_toolkit==4.0.0->-r requirements.txt (line 5))
  Using cached wcwidth-0.6.0-py3-none-any.whl.metadata (30 kB)
Collecting uniconfig==1.0.1 (from pytest==7.0.0->-r requirements.txt (line 6))
  Using cached uniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging==22 (from pytest==7.0.0->-r requirements.txt (line 6))
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pluggy<2,>=1.5 (from pytest==7.0.0->-r requirements.txt (line 6))
  Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.5 kB)
Collecting pygments==2.7.2 (from pytest==7.0.0->-r requirements.txt (line 6))
  Using cached pygments-2.19.2-py3-none-any.whl.metadata (2.5 kB)
Collecting coverage==7.10.6 (from coverage[tool]==7.10.6->pytest-cov==4.0.0->-r requirements.txt (line 7))
  Using cached coverage-7.13.4-cp310-cp313-manylinux1_x86_64-manylinux_2_28_x86_64-manylinux_2_5_x86_64.whl.metadata (8.5 kB)
Using cached pyyaml-6.0.3-cp310-cp313-manylinux2014_x86_64-manylinux_2_17_x86_64-manylinux_2_28_x86_64.whl (801 kB)
Using cached psutil-7.2.2-cp38-ab33-manylinux2010_x86_64-manylinux_2_12_x86_64-manylinux_2_28_x86_64.whl (155 kB)
Using cached pymysql-1.1.2-py3-none-any.whl (45 kB)
Using cached questionnaire-2.1.1-py3-none-any.whl (36 kB)
Using cached prompt_toolkit-3.0.52-py3-none-any.whl (391 kB)
Using cached pytest-9.0.2-py3-none-any.whl (374 kB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached pytest-cov-7.0.0-py3-none-any.whl (22 kB)
Using cached coverage-7.13.4-cp310-cp313-manylinux1_x86_64-manylinux_2_28_x86_64-manylinux_2_5_x86_64.whl (253 kB)
Using cached uniconfig-2.3.0-py3-none-any.whl (17.5 kB)
Using cached packaging-26.0-py3-none-any.whl (74 kB)
Using cached pygments-2.19.2-py3-none-any.whl (1.2 MB)
Using cached wcwidth-0.6.0-py3-none-any.whl (94 kB)
Installing collected packages: wcwidth, pyyaml, pymysql, pygments, psutil, pluggy, packaging, uniconfig, coverage, pytest, prompt_toolkit, questionnaire, pytest-cov
Successfully installed coverage-7.13.4 uniconfig-2.3.0 packaging-26.0 pluggy-1.6.0 prompt_toolkit-3.0.52 psutil-7.2.2 pygments-2.19.2 pymysql-1.1.2 pytest-9.0.2 pytest-cov-7.0.0 pyyaml-6.0.3 questionnaire-2.1.1 wcwidth-0.6.0
(venv) root@sql-1-MSPR-116
```

5.3 Configuration

Copier et editer le fichier de configuration :

```
cp config.example.yaml config.yaml
nano config.yaml # ou votre editeur prefere
```

IMPORTANT : Ne jamais committer config.yaml dans Git. Ce fichier contient des secrets (mots de passe, tokens).

 config.yaml U X config.yaml

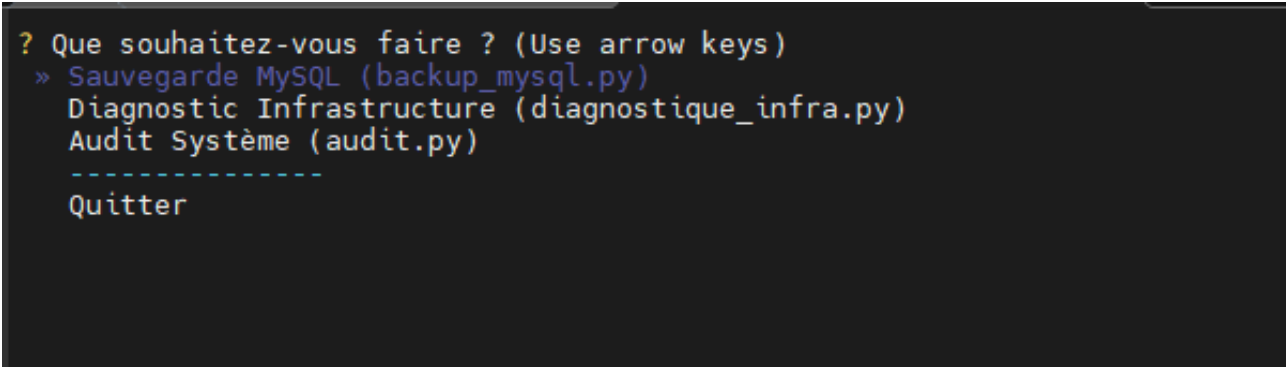
```
4  # MySQL Configuration
5  mysql:
6    host: "192.168.10.14"
7    user: "root"
8    password: "mspr25@"
9    port: 3306
10   backup_dir: "backups_mysql"
11
12  # Agent Configuration
13  agent:
14    port: 6000
15    auth_token: "mspr25@"
16    timeout: 5
17
18  # Diagnostic Configuration
19  diagnostic:
20    output_dir: "rapports_ntl"
21    servers:
22      - name: "ESXi Host"
23        ip: "10.10.10.97"
24        ports:
25          https_443: 443
26          ssh_22: 22
27        agent_enabled: true
28
29      - name: "Grafana"
30        ip: "192.168.10.13"
31        ports:
32          grafana_3000: 3000
33          ssh_22: 22
34        agent_enabled: true
35
36      - name: "WMS-DB"
37        ip: "192.168.10.14"
38        ports:
39          mysql_3306: 3306
40          ssh_22: 22
41        agent_enabled: true
42
43      - name: "Serveur AD 1"
44        ip: "192.168.10.10"
45        ports:
46          ldap_389: 389
47          ldaps_636: 636
48          dns_53: 53
49          kerberos_88: 88
50          rdp_3389: 3389
51        agent_enabled: true
52
53      - name: "Serveur AD MSPR"
54        ip: "192.168.10.11"
55        ports:
```

6. Guide d'Utilisation

6.1 Interface de selection (selecteur.py)

Le script selecteur.py fournit une interface interactive permettant de choisir le module a executer.

```
python3 selecteur.py
```



```
? Que souhaitez-vous faire ? (Use arrow keys)
» Sauvegarde MySQL (backup_mysql.py)
  Diagnostic Infrastructure (diagnostique_infra.py)
  Audit Système (audit.py)
  -----
  Quitter
```

6.2 Execution directe des modules

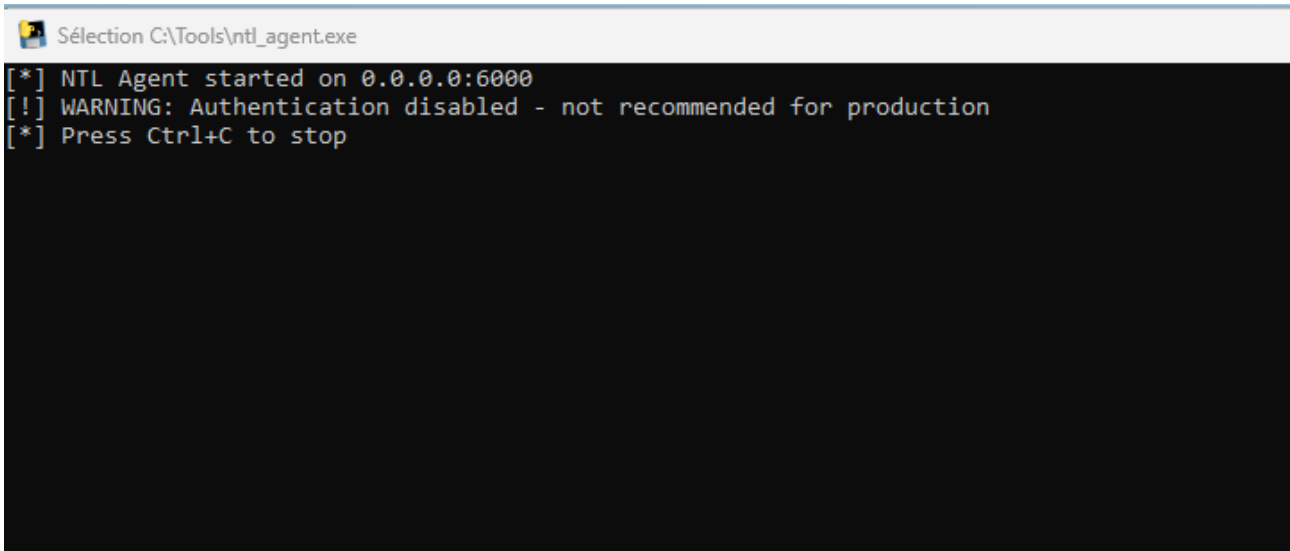
```
# Diagnostic infrastructure
python3 diagnostique_infra.py

# Sauvegarde MySQL
python3 backup_mysql.py

# Audit d'obsolescence
python3 audit.py
```

6.3 Deploiement de l'agent

```
# Sur chaque serveur a monitorer
export NTL_AGENT_TOKEN="votre_token_securise"
python3 ntl_agent.py --port 6000
```



```
Sélection C:\Tools\ntl_agent.exe
[*] NTL Agent started on 0.0.0.0:6000
[!] WARNING: Authentication disabled - not recommended for production
[*] Press Ctrl+C to stop
```

7. Securite

7.1 Authentification des agents

Les agents utilisent une authentification par token pour securiser les communications. Le token doit etre identique sur l'agent et dans la configuration centrale.

Generation d'un token securise

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

7.2 Bonnes pratiques

- Ne jamais stocker de mots de passe en clair dans le code source
- Utiliser config.yaml pour centraliser les secrets
- Ne pas exposer le port de l'agent (6000) directement sur Internet
- Deplacer les sauvegardes vers un stockage externe securise (NAS/Cloud)
- Utiliser des tokens d'authentification forts (32+ caracteres)
- Executer l'audit reseau avec les privileges root uniquement si necessaire

Le fichier config.yaml est inclus dans .gitignore pour eviter de committer des secrets.

8. Maintenance et Support

8.1 Codes de retour

Code	Signification	Action recommandee
0	Succes complet	Aucune action requise
1	Avertissements	Verifier les alertes dans le rapport
2	Erreur critique	Intervention immediate requise

8.2 Artefacts generes

Type	Emplacement	Format
Diagnostics	rapports_ntl/diagnostic_YYYYMMDD_HHMMSS.json	JSON
Backups	backups_mysql/backup_complet_YYYYMMDD_HHMMSS.sql	SQL
Audits	audit_eol_ntl_YYYYMMDD_HHMMSS.csv	CSV

8.3 Tests

```
# Executer les tests unitaires
pytest tests/ -v

# Avec rapport de couverture
pytest tests/ --cov=. --cov-report=html
```

```

create mode 100644 images/support_tool/0001Diagnostic.png
(venv) root@web:/home/web/MSPR-NTL# pytest tests/ --cov=. --cov-report=html
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/web/MSPR-NTL
plugins: cov-7.0.0
collected 36 items

tests/test_agent.py .....F..sFF.....
tests/test_config.py .....
tests/test_diagnostic.py .....

===== FAILURES =====
TestAgentProtocol.test_debug_command

self = <test_agent.TestAgentProtocol object at 0x7f71862f9370>, agent_server = <ntl_agent.NTLAgent object at 0x7f7185f53680>

    def test_debug_command(self, agent_server):
        """Test debug command returns full agent status"""
        client = AgentClient('127.0.0.1', 6001, auth_token='test_token_12345')
        debug_info = client.debug()

>       assert debug_info is not None
E       assert None is not None

tests/test_agent.py:162: AssertionError
----- Captured stdout setup -----
[!] Failed to start agents: [Errno 98] Address already in use
----- Captured stdout teardown -----

[*] Stopping NTL Agent...
[*] NTL Agent stopped
TestAgentClient.test_client_connection_timeout

self = <test_agent.TestAgentClient object at 0x7f718645f9d0>

    def test_client_connection_timeout(self):
        """Test client timeout on unreachable host"""
        result = query_agent('192.0.2.1', 6000, timeout=1)

        assert result['status'] == 'error'
>       assert 'timeout' in result['error'] or 'connection_failed' in result['error']
E       AssertionError: assert ('timeout' in 'Failed to retrieve metrics' or 'connection_failed' in 'Failed to retrieve metrics')

tests/test_agent.py:247: AssertionError
TestAgentClient.test_client_connection_refused

self = <test_agent.TestAgentClient object at 0x7f718645fb10>

    def test_client_connection_refused(self):
        """Test client handling of connection refused"""
        result = query_agent('127.0.0.1', 9999, timeout=1)

        assert result['status'] == 'error'
>       assert 'connection_failed' in result['error']
E       AssertionError: assert 'connection_failed' in 'Failed to retrieve metrics'

tests/test_agent.py:254: AssertionError
===== tests coverage =====
coverage: platform linux, python 3.13.5-final-0

Coverage HTML written to dir htmlcov
===== short test summary info =====
FAILED tests/test_agent.py::TestAgentProtocol::test_debug_command - assert None is not None
FAILED tests/test_agent.py::TestAgentClient::test_client_connection_timeout - AssertionError: assert ('timeout' in 'Failed to retrieve metrics' or 'connection_failed' in 'Failed to retrieve metrics')
FAILED tests/test_agent.py::TestAgentClient::test_client_connection_refused - AssertionError: assert 'connection_failed' in 'Failed to retrieve metrics'
1 failed, 34 passed, 1 skipped in 17.25s

```

9. Annexes

9.1 Structure du projet

```
MSPR-NTL/  
|-- agent_client.py      # Client pour interroger les agents  
|-- audit.py            # Module d'audit EOL  
|-- backup_mysql.py     # Module de sauvegarde MySQL  
|-- config_loader.py    # Chargement configuration YAML  
|-- config.example.yaml # Template de configuration  
|-- diagnostique_infra.py # Module de diagnostic  
|-- ntl_agent.py        # Agent daemon  
|-- selecteur.py        # Interface de selection  
|-- requirements.txt     # Dependances Python  
|-- README.md           # Documentation principale  
|-- documentation/      # Documentation detaillee  
|-- tests/              # Tests unitaires
```

9.2 Fichier requirements.txt

```
pyyaml>=6.0  
psutil>=5.9.0  
pymysql>=1.0.0  
questionary>=2.0.0  
pytest>=7.0.0  
pytest-cov>=4.0.0
```

9.3 Liens utiles

- Repository GitHub : <https://github.com/chtipilou/MSPR-NTL>
- Documentation Python psutil : <https://psutil.readthedocs.io/>
- Documentation PyYAML : <https://pyyaml.org/wiki/PyYAMLDocumentation>
- Guide systemd : <https://www.freedesktop.org/software/systemd/man/systemd.service.html>